



Formation 3 Jours — Jour 3

Modules 5, 6 & 7 · Données, API natives & Usages avancés



Sommaire — Jour 3

01 Formulaires & validation

02 Fetch, Axios & gestion des erreurs

03 Stockage local & offline

04 API natives React Native

05 Plugins Cordova & modules natifs

06 Best Practices & erreurs fréquentes

07 Tests unitaires & fonctionnels

08 Publier l'application

09 Mises à jour Over The Air (OTA)

10 Frameworks & outils complémentaires

11 TP — Formulaire, persistance, tests & déploiement

Module 5 · Formulaires & Données

Saisie, validation, fetch et persistance



Les composants de formulaire

```
import { TextInput, Switch, Slider } from 'react-native';
import { Picker } from '@react-native-picker/picker';

// TextInput – le composant central des formulaires
<TextInput
  value={email}
  onChangeText={setEmail}
  placeholder="votre@email.com"
  keyboardType="email-address"      // email-address | numeric | phone-pad | url
  autoCapitalize="none"            // none | sentences | words | characters
  autoCorrect={false}
  secureTextEntry={false}          // true pour mot de passe
  returnKeyType="next"             // next | done | search | send
  onSubmitEditing={() => passwordRef.current?.focus()}
  maxLength={100}
/>

// Switch – booléen
<Switch value={notifications} onChange={setNotifications}
  trackColor={{ true: '#0d47a1' }} thumbColor="#fff" />
```

✓ Validation avec React Hook Form

```
npm install react-hook-form @hookform/resolvers zod
```

```
import { useForm, Controller } from 'react-hook-form';
import { z } from 'zod';
import { zodResolver } from '@hookform/resolvers/zod';

const schema = z.object({
  email: z.string().email('Email invalide'),
  password: z.string().min(8, 'Minimum 8 caractères'),
  age: z.number().min(18, 'Vous devez avoir 18 ans'),
});

type FormData = z.infer<typeof schema>;

function MonFormulaire() {
  const { control, handleSubmit, formState: { errors } } = useForm<FormData>({
    resolver: zodResolver(schema),
    defaultValues: { email: '', password: '' },
  });

  const onSubmit = (data: FormData) => console.log(data);
}
```



Champs contrôlés avec React Hook Form

```
// Champ email contrôlé
<Controller
  control={control}
  name="email"
  render={({ field: { onChange, onBlur, value } }) => (
    <View>
      <TextInput
        value={value}
        onChangeText={onChange}
        onBlur={onBlur}
        placeholder="Email"
        keyboardType="email-address"
        style={[styles.input, errors.email && styles.inputError]}
      />
      {errors.email && (
        <Text style={styles.errorText}>{errors.email.message}</Text>
      )}
    </View>
  )}
/>

<TouchableOpacity onPress={handleSubmit(onSubmit)}>
  <Text>Valider</Text>
</TouchableOpacity>
```



Fetch API — Requêtes réseau

```
// Fetch natif – disponible sans installation
async function fetchUsers(): Promise<User[]> {
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), 10_000);

  try {
    const response = await fetch('https://api.example.com/users', {
      method: 'GET',
      headers: {
        'Content-Type': 'application/json',
        Authorization: `Bearer ${token}`,
      },
      signal: controller.signal,
    });

    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    return await response.json();
  } finally {
    clearTimeout(timeoutId);
  }
}
```

⚡ Axios — Client HTTP avancé

```
npm install axios
```

```
// lib/api.ts – Instance Axios configurée
import axios from 'axios';
import AsyncStorage from '@react-native-async-storage/async-storage';

const api = axios.create({
  baseURL: 'https://api.example.com',
  timeout: 10_000,
  headers: { 'Content-Type': 'application/json' },
});

// Intercepteur – ajoute le token automatiquement
api.interceptors.request.use(async (config) => {
  const token = await AsyncStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Intercepteur – gère les erreurs 401 (token expiré)
api.interceptors.response.use(null, async (error) => {
  if (error.response?.status === 401) await refreshToken();
  return Promise.reject(error);
});
```




Stockage local — AsyncStorage

```
npm install @react-native-async-storage/async-storage
```

```
import AsyncStorage from '@react-native-async-storage/async-storage';

// Écrire
await AsyncStorage.setItem('user', JSON.stringify({ id: 1, nom: 'Alice' }));
await AsyncStorage.multiSet([['theme', 'dark'], ['lang', 'fr']]);

// Lire
const raw = await AsyncStorage.getItem('user');
const user = raw ? JSON.parse(raw) : null;

// Supprimer
await AsyncStorage.removeItem('user');
await AsyncStorage.clear(); // ⚠️ Tout supprimer

// Lister les clés
const keys = await AsyncStorage.getAllKeys();
```

⚡ MMKV — Alternative ultra-rapide

```
npm install react-native-mmkv
```

```
import { MMKV } from 'react-native-mmkv';

// Synchrone ! – 10× plus rapide qu'AsyncStorage
const storage = new MMKV({ id: 'app-storage', encryptionKey: 'secret' });

// Écrire
storage.set('token', 'eyJ...');
storage.set('count', 42);
storage.set('settings', JSON.stringify({ darkMode: true }));

// Lire (synchrone – pas d'await)
const token = storage.getString('token');
const count = storage.getNumber('count');
const darkMode = JSON.parse(storage.getString('settings') ?? '{}').darkMode;

// MMKV est le choix recommandé pour les nouvelles apps en 2025
// AsyncStorage reste utile pour la compatibilité Expo Go
```



Expo SecureStore — Données sensibles

```
import * as SecureStore from 'expo-secure-store';

// Stocker un secret (Keychain iOS / Keystore Android)
await SecureStore.setItemAsync('jwt_token', token);
await SecureStore.setItemAsync('refresh_token', refreshToken, {
  keychainAccessible: SecureStore.WHEN_UNLOCKED,
});

// Lire
const token = await SecureStore.getItemAsync('jwt_token');

// Supprimer
await SecureStore.deleteItemAsync('jwt_token');

// ✅ Utiliser SecureStore pour :
//   tokens JWT, refresh tokens, clés API, données biométriques
// ❌ Ne jamais stocker ces données dans AsyncStorage ou MMKV
```



Gestion offline — NetInfo & WatermelonDB

```
npm install @react-native-community/netinfo
```

```
import NetInfo from '@react-native-community/netinfo';

// Vérifier la connexion
const state = await NetInfo.fetch();
console.log(state.isConnected, state.type); // wifi | cellular | none

// Écouter les changements
const unsubscribe = NetInfo.addEventListener(state => {
  if (!state.isConnected) showBanner('Mode hors ligne');
});

// WatermelonDB — base de données relationnelle locale (offline first)
// npm install @nozbe/watermelondb
// Idéal pour les apps avec synchronisation serveur complexe

// SQLite via Expo
import * as SQLite from 'expo-sqlite';
const db = await SQLite.openDatabaseAsync('monapp.db');
await db.execAsync('CREATE TABLE IF NOT EXISTS todos (id INTEGER PRIMARY KEY, text TEXT)');
```



Stratégies offline — Résumé

Besoin	Solution
Clés/valeurs simples	MMKV (synchrone, rapide)
Données sensibles	Expo SecureStore
Données relationnelles	Expo SQLite
Sync serveur complexe	WatermelonDB
Cache requêtes réseau	TanStack Query (persist)
Compatibilité Expo Go	AsyncStorage

Pattern recommandé : écrire en local d'abord (optimistic update), synchroniser avec le serveur ensuite.

Module 6 · Interagir avec le Terminal

API natives, plugins et modules natifs

Géolocalisation

```
import * as Location from 'expo-location';

async function obtenirPosition() {
  // Demander la permission
  const { status } = await Location.requestForegroundPermissionsAsync();
  if (status !== 'granted') {
    alert('Permission refusée');
    return;
  }

  // Position actuelle
  const position = await Location.getCurrentPositionAsync({
    accuracy: Location.Accuracy.High,
  });
  console.log(position.coords.latitude, position.coords.longitude);

  // Suivi en temps réel
  const sub = await Location.watchPositionAsync(
    { accuracy: Location.Accuracy.Balanced, TimeInterval: 5000 },
    (pos) => console.log(pos.coords)
  );
  return () => sub.remove(); // Cleanup
}
```



Caméra & Galerie

```
import { CameraView, useCameraPermissions } from 'expo-camera';
import * as ImagePicker from 'expo-image-picker';

// Accès caméra
function EcranCamera() {
  const [permission, requestPermission] = useCameraPermissions();
  const cameraRef = useRef<CameraView>(null);

  if (!permission?.granted) return <Button title="Autoriser" onPress={requestPermission} />;

  const prendrePicture = async () => {
    const photo = await cameraRef.current?.takePictureAsync({ quality: 0.8 });
    console.log(photo?.uri);
  };

  return <CameraView ref={cameraRef} style={{ flex: 1 }} facing="back" />;
}

// Sélecteur galerie
const result = await ImagePicker.launchImageLibraryAsync({
  mediaTypes: ['images'], allowsEditing: true, aspect: [1, 1], quality: 0.8,
});
if (!result.canceled) uploadImage(result.assets[0].uri);
```


Notifications Push

```
npx expo install expo-notifications expo-device
```

```
import * as Notifications from 'expo-notifications';
import * as Device from 'expo-device';

async function enregistrerPourNotifications() {
  if (!Device.isDevice) return; // Pas sur émulateur

  const { status } = await Notifications.requestPermissionsAsync();
  if (status !== 'granted') return;

  // Token Expo Push (pour backend)
  const token = (await Notifications.getExpoPushTokenAsync({
    projectId: Constants.expoConfig?.extra?.eas?.projectId,
  })).data;

  // Envoyer au backend
  await api.post('/push-token', { token, platform: Platform.OS });
}

// Notification locale immédiate
await Notifications.scheduleNotificationAsync({
  content: { title: 'RN Formation', body: 'Rappel : TP à rendre demain !' },
  trigger: null, // null = immédiat
});
```



Vibration, Capteurs & Autres API

```
import { Vibration, AppState, Linking, Share, Clipboard } from 'react-native';
import * as Haptics from 'expo-haptics';
import { Accelerometer } from 'expo-sensors';

// Haptics – retour tactile natif
await Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Medium);
await Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success);

// AppState – détecter foreground/background
AppState.addListener('change', state => {
  if (state === 'active') rechargerDonnees();
});

// Ouvrir une URL externe / deep link
await Linking.openURL('https://reactnative.dev');
await Linking.openURL('tel:+33612345678');
await Linking.openURL('mailto:contact@example.com');

// Partage natif iOS/Android
await Share.share({ message: 'Découvrez cette app !', url: 'https://...' });
```



Biométrie & Sécurité

```
import * as LocalAuthentication from 'expo-local-authentication';

async function authentifierBiometrie() {
  const supported = await LocalAuthentication.hasHardwareAsync();
  if (!supported) return false;

  const enrolled = await LocalAuthentication.isEnrolledAsync();
  if (!enrolled) return false;

  const result = await LocalAuthentication.authenticateAsync({
    promptMessage: 'Confirmer votre identité',
    cancelLabel: 'Annuler',
    fallbackLabel: 'Utiliser le code',
    disableDeviceFallback: false,
  });

  return result.success;
}
```

Modules natifs custom — Turbo Module

```
// Définir l'interface TypeScript (CodeGen)
// NativeModules/NativeMaModule.ts
import type { TurboModule } from 'react-native';
import { TurboModuleRegistry } from 'react-native';

export interface Spec extends TurboModule {
  getMessage(): string;
  setMessage(msg: string): void;
  getDeviceInfo(): Promise<{ model: string; os: string }>;
}

export default TurboModuleRegistry.getEnforcing<Spec>('NativeMaModule');
```

```
// iOS – Swift (MaModuleImpl.swift)
@objc(NativeMaModule)
class NativeMaModule: NSObject, NativeMaModuleSpec {
  @objc func getMessage() -> String { return "Hello depuis Swift !" }
}
```

Plugins Cordova / Capacitor avec React Native

- React Native privilégie ses propres modules natifs
- Certains plugins Cordova peuvent être wrappés via un bridge JS
- **Capacitor** est plus moderne — adapté à Ionic, pas React Native
- Pour React Native, préférer les modules de la communauté :
 - `react-native-camera` → `expo-camera` / `react-native-vision-camera`
 - `cordova-plugin-geolocation` → `expo-location`
 - `cordova-plugin-push` → `expo-notifications`

En 2025, l'écosystème Expo couvre 95%+ des besoins courants sans plugin tiers.

Module 7 · Usages Avancés

Best Practices, Tests, Publication & OTA

✓ Best Practices — Performance

```
// 1. Mémoïsation – éviter les re-renders inutiles
const ListItem = React.memo(({ item }: { item: Item }) => (
  <Text>{item.title}</Text>
));

// 2. useCallback pour les handlers passés en props
const handlePress = useCallback((id: string) => {
  dispatch(selectItem(id));
}, [dispatch]);

// 3. useMemo pour les calculs coûteux
const filtered = useMemo(
  () => items.filter(i => i.active).sort((a, b) => a.date - b.date),
  [items]
);

// 4. keyExtractor stable – jamais l'index
keyExtractor={(item) => item.id} // ✓
keyExtractor={(_, index) => index} // ✗

// 5. Éviter les objets inline dans style
// ✗ <View style={{ marginTop: 16 }} /> (nouvel objet à chaque render)
// ✓ <View style={styles.container} /> (référence stable)
```

⚠ Erreurs fréquentes à éviter

```
// ❌ Mutation directe du state
state.items.push(newItem);           // Jamais !
// ✅
setItems(prev => [...prev, newItem]);

// ❌ useEffect sans tableau de dépendances
useEffect(() => { fetchData(); });    // Boucle infinie !
// ✅
useEffect(() => { fetchData(); }, [userId]);

// ❌ Texte nu hors de <Text>
<View>Texte nu</View>                // Crash Android
// ✅
<View><Text>Texte</Text></View>

// ❌ Image sans dimensions
<Image source={{ uri: url }} />      // Ne s'affiche pas
// ✅
<Image source={{ uri: url }} style={{ width: 200, height: 200 }} />

// ❌ FlatList dans ScrollView
// → Jamais de FlatList dans une ScrollView (conflits de scroll)
```


Sécurité — Bonnes pratiques

```
// ✓ Ne jamais stocker de secrets dans le code source
// ✗ const API_KEY = 'sk-abc123...'; ← commit dans git = fuite
// ✓ Utiliser expo-constants + variables d'environnement EAS

// app.config.js
export default {
  extra: {
    apiUrl: process.env.API_URL, // Injecté par EAS Secrets
  },
};

// Dans le code
const apiUrl = Constants.expoConfig?.extra?.apiUrl;

// ✓ Tokens dans SecureStore (pas AsyncStorage)
await SecureStore.setItemAsync('access_token', token);

// ✓ Valider les inputs côté serveur – jamais uniquement côté client
// ✓ Certificate Pinning pour les APIs sensibles
// ✓ Obfuscation du bundle JS en production
```

Tests unitaires — Jest & Testing Library

```
npm install -D @testing-library/react-native @testing-library/jest-native
```

```
// __tests__/Bouton.test.tsx
import { render, fireEvent } from '@testing-library/react-native';
import Bouton from '../components/Bouton';

describe('Bouton', () => {
  it('affiche le titre', () => {
    const { getByText } = render(<Bouton titre="Valider" onPress={() => {}} />);
    expect(getByText('Valider')).toBeTruthy();
  });

  it('appelle onPress au tap', () => {
    const onPress = jest.fn();
    const { getByText } = render(<Bouton titre="OK" onPress={onPress} />);
    fireEvent.press(getByText('OK'));
    expect(onPress).toHaveBeenCalledTimes(1);
  });
});

// Lancer les tests
npx jest --watchAll --coverage
```



Tests de Hooks & Store Redux

```
// __tests__/useCompteur.test.ts
import { renderHook, act } from '@testing-library/react-native';
import { useCompteur } from '../hooks/useCompteur';

test('useCompteur - incrément', () => {
  const { result } = renderHook(() => useCompteur());
  expect(result.current.count).toBe(0);
  act(() => result.current.incremente());
  expect(result.current.count).toBe(1);
});

// Test d'un reducer Redux
import todosReducer, { ajouter } from '../store/todosSlice';

test('reducer todos - ajouter', () => {
  const state = todosReducer([], ajouter('Test'));
  expect(state).toHaveLength(1);
  expect(state[0].text).toBe('Test');
  expect(state[0].done).toBe(false);
});
```



Tests fonctionnels — Maestro & Detox

```
# Maestro – test E2E déclaratif (YAML)
# .maestro/login.yaml
appId: com.monapp
---
- launchApp
- tapOn: "Email"
- inputText: "test@example.com"
- tapOn: "Mot de passe"
- inputText: "password123"
- tapOn: "Se connecter"
- assertVisible: "Bienvenue"
```

```
# Lancer les tests Maestro
maestro test .maestro/login.yaml

# Detox – tests E2E JavaScript
npx detox test --configuration ios.sim.debug
```

Publier sur l'App Store & Play Store

1. Configurer app.json (version, bundleIdentifier, package)

```
{
  "expo": {
    "version": "1.0.0",
    "ios": { "bundleIdentifier": "com.societe.monapp" },
    "android": { "package": "com.societe.monapp" }
  }
}
```

2. EAS Build – build de production

```
npx eas build --platform ios --profile production
```

```
npx eas build --platform android --profile production
```

3. Soumettre aux stores

```
npx eas submit --platform ios      # → App Store Connect
```

```
npx eas submit --platform android  # → Google Play Console
```

iOS (App Store Connect) :

- Compte Apple Developer (99 \$/an)
- Certificats et provisioning profiles gérés par EAS
- Captures d'écran pour chaque taille d'iPhone/iPad
- Review Apple : 24–48h en moyenne (2025)

Android (Google Play Console) :

- Compte développeur (25 \$ unique)
- APK signé ou App Bundle (AAB) — AAB recommandé
- Captures d'écran, description, politique de confidentialité
- Review : quelques heures à 3 jours



Mises à jour Over-The-Air (OTA)

```
# EAS Update – modifier le JS/assets SANS repasser par les stores

# Publier une mise à jour
npx eas update --branch production --message "Fix: crash liste vide"

# Branches de déploiement
npx eas update --branch staging      # Tests internes
npx eas update --branch production # Utilisateurs finaux
```

```
// expo-updates – vérification au démarrage
import * as Updates from 'expo-updates';

async function verifierMajOTA() {
  const update = await Updates.checkForUpdateAsync();
  if (update.isAvailable) {
    await Updates.fetchUpdateAsync();
    await Updates.reloadAsync(); // Redémarrer l'app
  }
}
```

⚠ *OTA ne peut modifier que le **bundle JS** — les changements natifs (nouveaux modules) nécessitent un rebuild complet.*

Expo Application Services (EAS) — Vue d'ensemble

EAS Build → Builds cloud iOS & Android sans Mac local
EAS Submit → Soumission automatisée aux stores
EAS Update → OTA – mise à jour du bundle JS en production
EAS Metadata → Gestion des métadonnées des stores (ASO)
EAS Insights → Analytics de builds et performances

- **Plan Free** : 30 builds/mois, OTA illimité
- **Plan Production** : builds prioritaires, builds concurrents
- Intégration CI/CD GitHub Actions / GitLab CI



Frameworks complémentaires 2025

UI & Design Systems :

- **React Native Paper** — Material Design 3
- **Gluestack UI v2** — accessible, customisable, RSC-ready
- **NativeWind 4** — Tailwind CSS pour React Native

Performance & Monitoring :

- **Sentry** — crash reporting en temps réel
- **Datadog / Firebase Crashlytics** — monitoring production
- **Flashlight** — benchmarking performance mobile

Outils :

- **Storybook for RN** — développement de composants isolés
- **React Native Skia** — graphiques et effets visuels GPU
- **Legend List** — FlatList ultra-performante



React Native pour le Web

```
# React Native Web – partager du code entre mobile et web  
npm install react-native-web
```

```
# Expo – support web intégré  
npx expo start --web
```

```
# Composants compatibles web  
<View> → <div>  
<Text> → <span>  
<Image> → <img>
```

```
# Navigation – Expo Router supporte le web nativement  
# app/index.tsx est accessible sur / en web ET mobile
```

```
# Points d'attention  
# - StyleSheet : pas de Flexbox gap < RN 0.71 sur web  
# - Modules natifs : conditionnels avec Platform.OS  
# - Images : formats différents (SVG, WebP)
```



React Native et l'IA — Tendances 2025

- LLM on-device : Meta LLaMA 3 via `react-native-executorch` (Meta 2025)
- TensorFlow Lite : modèles embarqués pour classification, NLP
- Core ML (iOS) / ML Kit (Android) via modules Expo
- Vision Camera : analyse d'image temps réel, détection d'objets, OCR
- Whisper.cpp : transcription vocale locale (React Native port)

```
// Exemple : classification d'image avec ML Kit (Expo)
import * as ImageClassifier from 'expo-image-classifier';

const result = await ImageClassifier.classifyAsync(imageUri);
// → [{ label: 'chat', confidence: 0.92 }, ...]
```

TP — Jour 3

Objectif : Formulaire complet, persistance, tests et déploiement



TP — Formulaire de profil avec validation

```
// Formulaire complet : react-hook-form + zod
import { z } from 'zod';

const profilSchema = z.object({
  prenom: z.string().min(2, 'Minimum 2 caractères'),
  email: z.string().email('Email invalide'),
  telephone: z.string().regex(/^(\+33|0)[1-9](\d{8})$/, 'Téléphone invalide'),
  age: z.coerce.number().min(18, 'Majeur requis').max(120),
  newsletter: z.boolean(),
});

type ProfilForm = z.infer<typeof profilSchema>;

// Persistance avec MMKV ou AsyncStorage
const sauvegarder = async (data: ProfilForm) => {
  await AsyncStorage.setItem('profil', JSON.stringify(data));
};

const charger = async (): Promise<Partial<ProfilForm>> => {
  const raw = await AsyncStorage.getItem('profil');
  return raw ? JSON.parse(raw) : {};
};
```



TP — Fetch avec gestion d'erreurs

```
// Hook personnalisé – pattern réutilisable
function useFetch<T>(url: string) {
  const [data, setData] = useState<T | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let active = true;
    setLoading(true);
    setError(null);

    fetch(url)
      .then(r => { if (!r.ok) throw new Error(`HTTP ${r.status}`); return r.json(); })
      .then(d => { if (active) setData(d); })
      .catch(e => { if (active) setError(e.message); })
      .finally(() => { if (active) setLoading(false); });

    return () => { active = false; };
  }, [url]);

  return { data, loading, error };
}
```



TP — Tests unitaires du formulaire

```
// __tests__/FormulaireProfile.test.tsx
import { render, fireEvent, waitFor } from '@testing-library/react-native';
import FormulaireProfile from '../screens/FormulaireProfile';

describe('FormulaireProfile', () => {
  it('affiche une erreur pour un email invalide', async () => {
    const { getByPlaceholderText, getByText } = render(<FormulaireProfile />);

    fireEvent.changeText(getByPlaceholderText('Email'), 'invalide');
    fireEvent.press(getByText('Sauvegarder'));

    await waitFor(() => {
      expect(getByText('Email invalide')).toBeTruthy();
    });
  });

  it('soumet les données valides', async () => {
    const onSubmit = jest.fn();
    const { getByPlaceholderText, getByText } = render(
      <FormulaireProfile onSubmit={onSubmit} />
    );
    fireEvent.changeText(getByPlaceholderText('Email'), 'test@ok.com');
    fireEvent.press(getByText('Sauvegarder'));
    await waitFor(() => expect(onSubmit).toHaveBeenCalled());
  });
});
```



TP — Préparer l'app pour la production

```
# 1. Vérifier la configuration app.json / app.config.js
#   version, bundleIdentifier (iOS), package (Android)

# 2. Configurer EAS
npx eas build:configure

# 3. Variables d'environnement sécurisées
npx eas secret:create --name API_URL --value "https://api.prod.com"

# 4. Build de production
npx eas build --platform all --profile production

# 5. Tester le build de production sur émulateur
npx expo run:android --variant release

# 6. Publier une mise à jour OTA de test
npx eas update --branch staging --message "TP Jour 3 – version test"
```


? Questions de révision — Jour 3

- Pourquoi `react-hook-form` est-il recommandé plutôt qu'un state manuel pour les formulaires ?
- Quelle est la différence entre MMKV et AsyncStorage ? Quand utiliser l'un ou l'autre ?
- Qu'est-ce qu'une mise à jour OTA et quelles sont ses limites ?
- Comment gérer un token d'authentification de manière sécurisée ?
- Quand utiliser `React.memo`, `useCallback` et `useMemo` ?
- Quelle est la différence entre EAS Build et EAS Update ?

Pour aller plus loin

- **Documentation officielle** : reactnative.dev & docs.expo.dev
- **Expo SDK** : toutes les API natives disponibles
- **React Navigation** : reactnavigation.org
- **Communauté** : Discord React Native, [r/reactnative](https://r.reactnative.com)
- **Conférences** : App.js Conf, Chain React, React Native EU
- **Newsletters** : This Week in React, Expo Newsletter
- **Cours** : [William Candillon \(can-it-be-done-in-react-native\)](#), Expo blog

Formation React Native terminée

